

Reviewer Pack — Minimal Number of Self-Dual Codes and SAT Clause Subset Entr...

Entry 38f40fce7be0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 17:53:11 UTC

Minimal Number of Self-Dual Codes and SAT Clause Subset Entropy Correlation

Entry ID: 38f40fce7be0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 17:53:11 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Coding Theory (Self-Dual Codes) **Field B** (complexity object): Boolean Satisfiability (SAT) Clause Subsets

Statement:

For all n -variables SAT instances with clause subsets S , the minimal number of self-dual codes required to encode the clauses in S is linearly correlated with the entropy of S , such that $|S| = \Theta(2^{H(S)})$ for some constant H .

Rationale (proposer's reasoning):

Self-dual codes are a classical tool in coding theory known for their symmetry properties. Their application to SAT clause subsets could reveal a hidden relationship between the algebraic structure of clauses and their information content. This connection, if true, might expose novel insights into both areas.

Taxonomy category: c003b_cumulative_entropy/SC2#c49 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4d28530b7062fc27

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the number of self-dual codes (N) and entropy (H) across all seeds is greater than or equal to 0.8, with no seed's N exceeding the mean by more than 3 standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'self-dual code' AND 'Boolean satisfiability problem' - 'algebraic coding theory' AND 'SAT clause subsets' - 'entropy correlation' AND 'encoding self-dual codes'

Top relevant hits considered: - [http://arxiv.org/abs/1512.07347v1] Galois Self-Dual Constacyclic Codes - [http://arxiv.org/abs/quant-ph/0503236v1] On Self-Dual Quantum Codes, Graphs, and Boolean Functions - [http://arxiv.org/abs/1203.1527v4] Optimum Subcodes of Self-Dual Codes and Their Optimum Distance Profiles - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/1301.6236v2] Multi-Trial Guruswami-Sudan Decoding for Generalised Reed-Solomon Codes - [http://arxiv.org/abs/2605.09347v1] Dsat: A Native SAT Solver for Discrete Logic - [http://arxiv.org/abs/gr-qc/0003089v1] A Spacetime Foam Approach to the Schwarzschild-de Sitter Entropy - [http://arxiv.org/abs/1612.05917v1] Linear Quantum Entropy and Non-Hermitian Hamiltonians

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n):
        clauses = set()
        for _ in range(2**n):
            clause = tuple(sorted(random.sample(range(-n, n+1), 3)))
            if all(abs(lit) != abs(clause[0]) for lit in clause[1:]):
                clauses.add(clause)
        return clauses

    def self_dual_codes(clauses):
        n = max(abs(lit) for clause in clauses for lit in clause)
        codes = []
        for _ in range(len(clauses)):
            code = [0] * (2*n)
            for literal in random.sample(range(-n, n+1), 2*n):
                if literal not in code:
                    code[literal - 1] = 1
            codes.append(code)
        return codes, None
```

```

def entropy(clauses):
    num_clauses = len(clauses)
    total_bits = sum(math.log2(2**n) for clause in clauses)
    avg_bits_per_clause = total_bits / num_clauses
    return avg_bits_per_clause

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    clauses = generate_sat_instance(n)
    codes, error = self_dual_codes(clauses)
    if error is not None:
        return {
            "metric_name": "correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": f"mapping_undefined for n={n}"
        }
    results.append(len(codes))

mean_codes = sum(results) / len(results)
std_codes = math.sqrt(sum((x - mean_codes)**2 for x in results) / len(results))
correlation = None

if len(results) > 1:
    mean_entropy = entropy(clauses)
    covariance = sum((results[i] - mean_codes) * (i + 5 - mean_entropy) for i in range(len(results)))
    variance_entropy = sum((i + 5 - mean_entropy)**2 for i in range(len(results))) / (len(results) - 1)
    correlation = covariance / math.sqrt(variance_entropy * std_codes**2)

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation is not None and abs(correlation) >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        if trial_result["conjecture_holds"]:
            results.append(trial_result["metric_value"])

    mean_value = sum(results) / len(results)
    std_value = math.sqrt(sum((x - mean_value)**2 for x in results) / len(results))

```

```

support_fraction = len([r for r in results if abs(r) >= 0.8 * mean_value]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(abs(r) < 0.8 * mean_value for r in results):
    first_failing_seed = seeds[results.index(next(r for r in results if abs(r) < 0.8 * mean_value))]
    print(f"RESULT: FALSIFIED counterexample=\"correlation_less_than_0.8\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which prevents us from evaluating the Pearson correlation coefficient and standard deviation as required by the support condition. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14186
2	propose	ollama_remote	glm4:latest	0	0	13115
3	propose	ollama_remote	glm4:latest	0	0	12181
4	preregistration	ollama_remote	glm4:latest	0	0	10403
5	novelty	ollama_remote	glm4:latest	0	0	15962
6	novelty	ollama_remote	glm4:latest	0	0	10081
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22596
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10013
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14276
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11716
11	judge	ollama_remote	glm4:latest	0	0	11644

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146174 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/38f40fce7be0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/38f40fce7be0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/38f40fce7be0.tar.gz` (if generated)